1. Fp64 branch-free study: run records (runs started 20261016T2319Z)

Kind: benchmark runs, descriptive report (observations only; the interpretation is in the study report `../../report.pdf`).
Study: `research/fp64_branchfree_20261016/`, protocol `PROTOCOL.md` frozen at the protocol commit `41a7a585c` (`runs/protocol_commit.txt`) before the first run.
Commands: the protocol's "Commands" section, verbatim. Each run is `hwcost measure --fields fp64,fp64cm,pf64,w1 --prime-fast-rho-bits 44` (7 repetitions of about 100 ms; batches 16, 64, 256, 1024, 4096, 16384, 65536, 262144, 1048576, 4194304). Every run exited 0 and completed every arm; no arm failed or timed out.

1.1. Runs and binaries

run	started / finished UTC	binary SHA-256 (first 16)	source
base-1	20261016T2319Z / 20261016T2321Z	`e0ee7f6069742fb5`	protocol commit 41a7a585cae0dfcb6fd44b6bf504a02a21702be3 with src/cryptanalysis/fp_backend.rs and src/cryptanalysis/prime_fast.rs at f25d5e520
cand-1	20261016T2321Z / 20261016T2323Z	`5c7a0eaa6ba18225`	protocol commit 41a7a585cae0dfcb6fd44b6bf504a02a21702be3
base-2	20261016T2323Z / 20261016T2324Z	`e0ee7f6069742fb5`	protocol commit 41a7a585cae0dfcb6fd44b6bf504a02a21702be3 with src/cryptanalysis/fp_backend.rs and src/cryptanalysis/prime_fast.rs at f25d5e520
cand-2	20261016T2324Z / 20261016T2326Z	`5c7a0eaa6ba18225`	protocol commit 41a7a585cae0dfcb6fd44b6bf504a02a21702be3

Both binaries: `cargo build --release --bin hwcost` (default x86-64 target; compiled target features: none probed).

1.2. Host and isolation

- CPU: Intel(R) Xeon(R) Processor @ 2.10GHz (GenuineIntel, signature 6/207/2), 4 logical CPUs; virtualization: cpu_flag = hypervisor, systemd_detect_virt_vm = kvm kernel 6.18.44-fc-v139 rustc 1.97.0 (2d8144b78 2026-07-07).
- Isolation **L0**: shared cloud container, no pinning, no frequency control; arms interleaved, order rotated per repetition. No other build or test ran during the four runs.
- **No hardware performance counters** (guest kernel: no PMU driver for this CPU): branch misses are missing data, not zero.

1.3. Correctness gates

Every run's `stderr.txt` opens with one line per field: the batched walk equals the unbatched reference and stays on the curve, and the timed arithmetic's walk equals `fp_backend::Fp64`'s at the same prime point for point (for `pf64`, `prime_fast::Fp64` and `fp_backend::Fp64` share the Montgomery form). All passed in all four runs. Every `pfrho` sample is a known-answer solve whose logarithm was asserted. Before the runs, the targeted unit tests (92, including both new bit-identity tests) passed on the protocol commit.

1.4. Claims (computed by `hwcost compare` and `hwcost pair`)

id	statement	value	threshold	verdict
P1	max(r1, r2) over 6 arms kernel.fp64.* (batch ≥ 4096)	0.4363	0.85	PASS
P1pf	max(r1, r2) over 6 arms kernel.pf64.* (batch ≥ 4096)	0.4655	0.85	PASS
P4	max(r1, r2) over 1 arms pfrho.44* (batch ≥ 0)	0.7212	0.95	PASS
P2a	max(r1, r2) over 1 arms fp64.mul.latency* (batch ≥ 0)	1.001	1.1	PASS
P2b	max(r1, r2) over 1 arms fp64.sqr.latency* (batch ≥ 0)	0.9968	1.1	PASS
P2c	max(r1, r2) over 1 arms pf64.mul.latency* (batch ≥ 0)	1.0184	1.1	PASS
P2d	max(r1, r2) over 1 arms pf64.sqr.latency* (batch ≥ 0)	0.9978	1.1	PASS
C1a	max |r_k − 1| over 6 arms kernel.fp64cm.* (batch ≥ 4096)	0.0247	0.1	PASS
C1b	max |r_k − 1| over 6 arms kernel.w1.* (batch ≥ 4096)	0.0299	0.1	PASS
P3	worst fp64/fp64cm kernel ratio at B ≥ 4096 over 2 runs (12 cells)	1.0269	1.1	PASS

Counts of arm classes (candidate vs baseline, effect called only beyond max(2·drift, 2%) on the same side in both pairs):

group	class	arms
clock	no detected effect	2
fp64	no detected effect	8
fp64cm	candidate faster	2
fp64cm	no detected effect	6
kernel	candidate faster	21
kernel	no detected effect	19
mem	no detected effect	20
pf64	candidate slower	1

pf64	no detected effect	7
pfrho	candidate faster	1
scaling	candidate faster	6
scaling	no detected effect	6
w1	candidate faster	1
w1	no detected effect	7

1.5. Rho kernel, B ≥ 4096 (ns per iteration, medians of 7)

arm	A1	B1	A2	B2	r1	r2	drift	margin	class
fp64.1048576	50.1	21.9	50.1	21.7	0.436	0.433	0	0.02	candidate faster
fp64.16384	49.8	19.8	49.7	20	0.398	0.402	0.002	0.02	candidate faster
fp64.262144	51	21.3	48.7	20.2	0.417	0.416	0.046	0.092	candidate faster
fp64.4096	47.4	20.2	47.7	20.1	0.426	0.422	0.008	0.02	candidate faster
fp64.4194304	50.8	21.7	51.9	22	0.428	0.424	0.02	0.04	candidate faster
fp64.65536	49.6	20	50.4	20.3	0.404	0.403	0.016	0.032	candidate faster
fp64cm.1048576	21.6	21.5	21.8	21.5	0.997	0.987	0.011	0.022	no detected effect
fp64cm.16384	20.6	20.2	20	20.1	0.979	1.001	0.028	0.055	no detected effect
fp64cm.262144	21	20.7	20.5	20.5	0.989	1	0.023	0.046	no detected effect
fp64cm.4096	20	19.7	20	19.9	0.983	0.999	0.002	0.02	no detected effect
fp64cm.4194304	21.8	22.2	22.5	22.3	1.019	0.991	0.033	0.067	no detected effect
fp64cm.65536	20.3	19.8	20.4	19.9	0.978	0.975	0.005	0.02	candidate faster
pf64.1048576	50.2	22.6	49.8	22.7	0.449	0.455	0.009	0.02	candidate faster
pf64.16384	47	21.2	46.5	21.4	0.452	0.46	0.01	0.021	candidate faster
pf64.262144	47.9	21.9	47.4	22.1	0.457	0.466	0.01	0.02	candidate faster
pf64.4096	47.2	21.4	47.3	21.3	0.454	0.45	0.004	0.02	candidate faster
pf64.4194304	50.4	23.4	50.1	23.1	0.465	0.461	0.006	0.02	candidate faster
pf64.65536	47.9	21.4	46.9	21.6	0.447	0.46	0.021	0.041	candidate faster
w1.1048576	31.9	31.9	31.8	31.3	1	0.986	0.004	0.02	no detected effect
w1.16384	30.7	29.9	29.9	30	0.975	1.005	0.026	0.053	no detected effect
w1.262144	30.6	31	30.7	30.7	1.014	1.002	0.002	0.02	no detected effect
w1.4096	30.3	30.2	30.1	30	0.994	0.997	0.008	0.02	no detected effect
w1.4194304	32.5	31.6	32	32	0.97	1	0.016	0.032	no detected effect
w1.65536	30.4	30.4	30.7	30.1	0.998	0.979	0.009	0.02	no detected effect

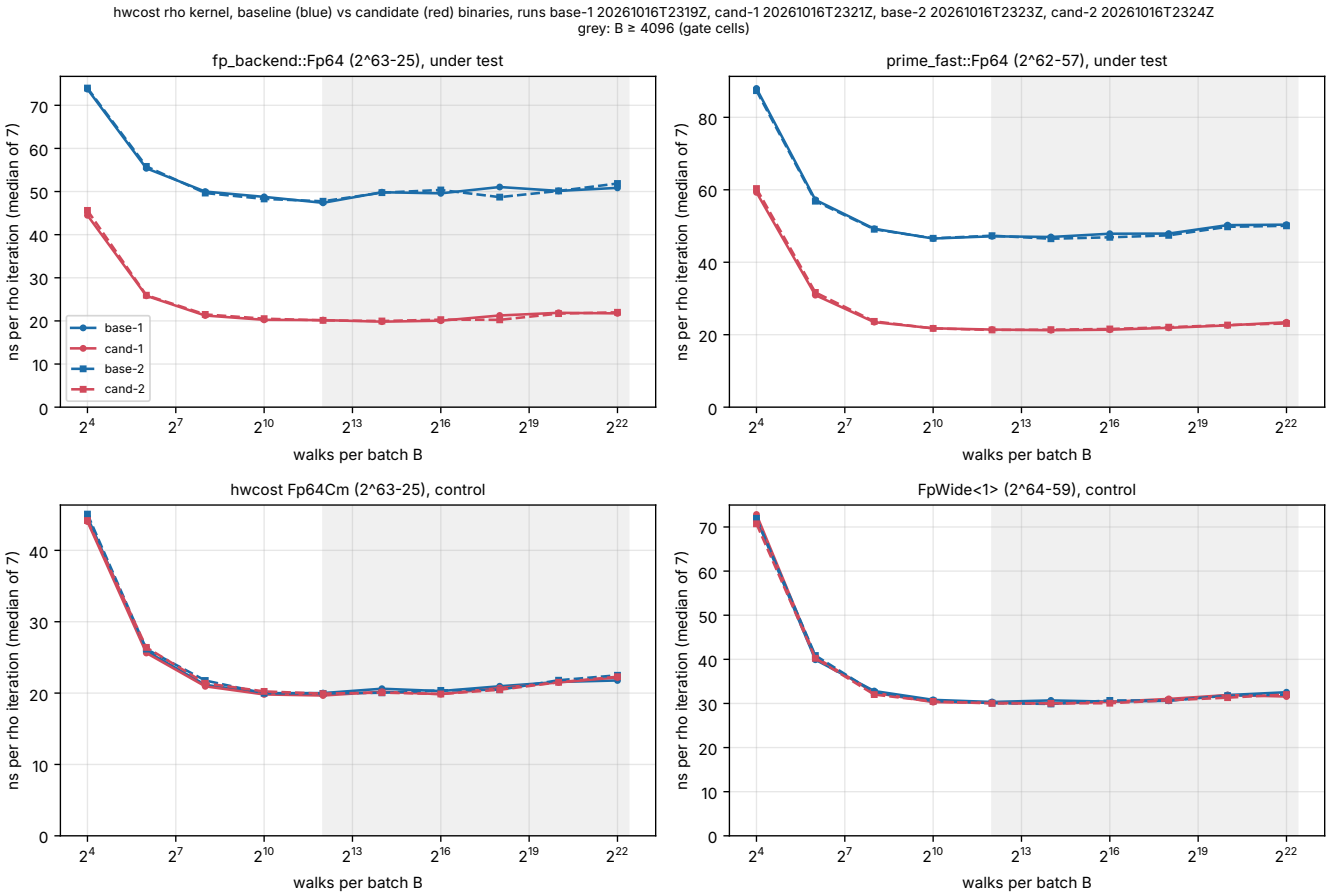


Figure 1: Rho kernel time per field against batch size in all four runs (medians of 7 repetitions).

1.6. Field operations and the production rho

arm	A1	B1	A2	B2	r1	r2	drift	margin	class
fp64.add.latency	0.82	1.09	0.91	1.08	1.328	1.178	0.119	0.238	no detected effect
fp64.add.throughput	0.47	0.47	0.48	0.47	0.994	0.978	0.023	0.047	no detected effect
fp64.inv.latency	406.57	399.47	422.43	400.53	0.983	0.948	0.039	0.078	no detected effect
fp64.inv.throughput	407.02	396.08	416.63	394.42	0.973	0.947	0.024	0.047	no detected effect
fp64.mul.latency	6.86	6.77	6.8	6.8	0.988	1.001	0.009	0.02	no detected effect
fp64.mul.throughput	1.38	1.37	1.41	1.36	1	0.967	0.025	0.051	no detected effect
fp64.sqr.latency	5.03	4.98	5	4.98	0.989	0.997	0.008	0.02	no detected effect
fp64.sqr.throughput	1.29	1.3	1.32	1.29	1.004	0.981	0.017	0.035	no detected effect
pf64.add.latency	0.88	1.12	0.87	1.1	1.264	1.27	0.016	0.033	candidate slower
pf64.add.throughput	0.48	0.47	0.47	0.49	0.978	1.051	0.026	0.053	no detected effect
pf64.inv.latency	631.15	619.64	619.86	609.33	0.982	0.983	0.018	0.036	no detected effect
pf64.inv.throughput	678.52	685.91	671.74	662.83	1.011	0.987	0.01	0.02	no detected effect
pf64.mul.latency	7.19	7.14	7.16	7.29	0.993	1.018	0.003	0.02	no detected effect
pf64.mul.throughput	1.48	1.46	1.45	1.5	0.987	1.036	0.018	0.036	no detected effect
pf64.sqr.latency	5.37	5.36	5.41	5.4	0.997	0.998	0.007	0.02	no detected effect
pf64.sqr.throughput	1.38	1.31	1.37	1.35	0.953	0.985	0.007	0.02	no detected effect
pfrho.44	98.88	68.15	98.61	71.12	0.689	0.721	0.003	0.02	candidate faster

Units: field operations ns per operation; `pfrho.44` ns per walk step of `prime_fast::rho_parallel` (one thread, one known-answer solve per sample). Every `pfrho.44` sample (ns per step):

run	samples (repetition order)
base-1	98.88, 93.49, 101.79, 131.98, 135.12, 94.17, 88.84
cand-1	66.95, 59.19, 75.43, 93.41, 93.35, 68.15, 57.1
base-2	98.61, 93.5, 105.48, 128.74, 127.58, 98.12, 91.55

cand-2	75.24, 65.08, 71.12, 93.58, 101.14, 64.12, 57.63

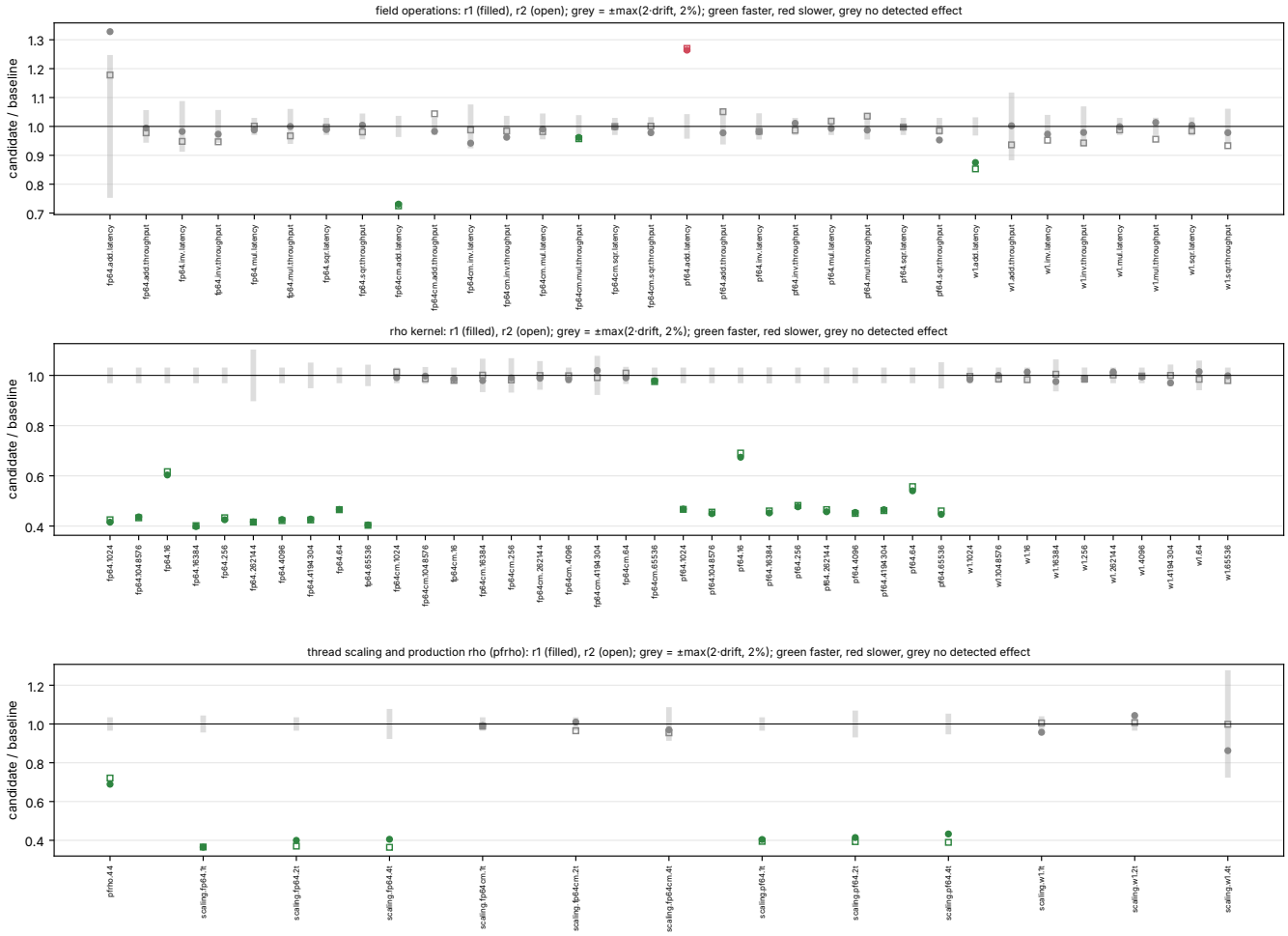


Figure 2: Candidate / baseline ratio for every arm, r_1 and r_2, with the drift margin.

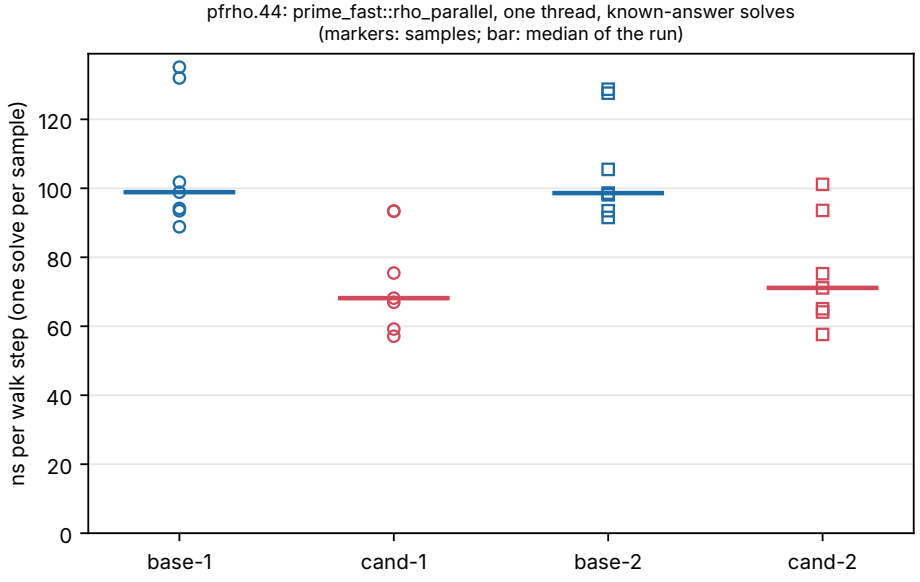


Figure 3: `pfrho.44`: every sample of the production rho in each run.

1.7. `fp64` against `fp64cm` in the candidate runs (P3)

run	B	fp64cm ns	fp64 ns	fp64 / fp64cm
cand-1	4096	19.66	20.19	1.0269
cand-1	16384	20.19	19.82	0.9821
cand-1	65536	19.85	20.05	1.0102

cand-1	262144	20.72	21.26	1.0264
cand-1	1048576	21.5	21.88	1.0179
cand-1	4194304	22.19	21.75	0.9799
cand-2	4096	19.93	20.13	1.0101
cand-2	16384	20.06	19.98	0.9958
cand-2	65536	19.87	20.29	1.0211
cand-2	262144	20.47	20.25	0.9894
cand-2	1048576	21.51	21.69	1.0082
cand-2	4194304	22.29	22	0.9868

1.8. Disassembly (D2)

binary	function	instructions	cond. jumps	cmov	mulx
base	`step_fp64`	303	32	4	0
base	`step_fp64cm`	285	20	12	0
cand	`step_fp64`	285	20	12	0
cand	`step_fp64cm`	285	20	12	0

`step_pf64` is linked in neither binary (`disasm.sh` reports it absent: no caller), so `prime_fast::rho_worker`, the production rho's per-thread loop, is counted instead (`disasm_rho_worker.sh`, whole symbol by its `nm -S` size):

binary	function	instructions	cond. jumps	cmov	mulx
base	`prime_fast::rho_worker`	1309	119	18	0
base	`prime_fast::rho_worker::{closure#0}`	210	21	0	0
cand	`prime_fast::rho_worker`	1285	107	30	0
cand	`prime_fast::rho_worker::{closure#0}`	210	21	0	0

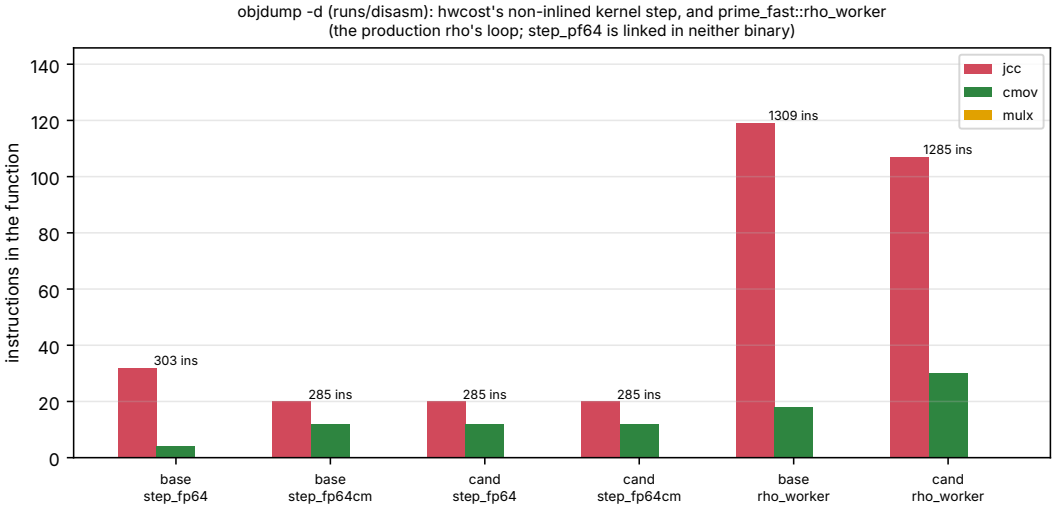


Figure 4: Instruction counts of the non-inlined kernel step and of `prime_fast::rho_worker` in each binary (`runs/disasm`).

1.9. Validation of the cost model per run (descriptive)

claim	base-1	cand-1	base-2	cand-2
C0	0.0063 PASS	0.0033 PASS	0.0029 PASS	0.0011 PASS
V1.fp64	0.0293 PASS	0.0481 PASS	0.0351 PASS	0.0086 PASS
V1.fp64cm	0.0774 PASS	0.0532 PASS	0.0318 PASS	0.0282 PASS
V1.pf64	0.0154 PASS	0.0289 PASS	0.0104 PASS	0.0214 PASS
V1.w1	0.0292 PASS	0.0185 PASS	0.0338 PASS	0.0219 PASS
V2.fp64	0.5496 FAIL	0.0836 PASS	0.5423 FAIL	0.0788 PASS
V2.fp64cm	0.0823 PASS	0.0777 PASS	0.0936 PASS	0.084 PASS
V2.pf64	0.508 FAIL	0.0576 PASS	0.5101 FAIL	0.0716 PASS
V2.w1	0.013 PASS	0.0141 PASS	0.0179 PASS	0.0131 PASS

1.10. Raw records

`runs/{base,cand}-{1,2}/` (`measurements.json`, `samples.csv`, `medians.csv`, `validation.{json,csv}`, `profile.json`, `fit.txt`, `stderr.txt`, `binary.txt`, timestamps), `runs/compare/`, `runs/cm-pair/`, `runs/disasm/`, `runs/protocol_commit.txt`. Graph sources: `../../plot.py` (layout only).